

DETAILED PROJECT REPORT

Scholar-Forge

A modular, menu-driven Python console application

To-Do List | Subject Performance | Book Tracker | Personal Curriculum

Submitted by	Vaidehi Singh
Registration No.	26BCE10385
Course	CSE1021 – Introduction to Problem Solving
Institution	VIT Bhopal
Faculty	Abha Sharma
Date of Submission	29/09/2026
GitHub Repository	https://github.com/vaidehi26bce10385-hub/Scholar-Forge

Contents

- 1. Cover Page**
- 2. Introduction**
- 3. Problem Statement**
- 4. Functional Requirements**
- 5. Non-functional Requirements**
- 6. System Architecture**
- 7. Design Diagrams**
 - 7.1 Use Case Diagram
 - 7.2 Workflow Diagram
 - 7.3 Sequence Diagram
 - 7.4 Class / Component Diagram
 - 7.5 ER Diagram
- 8. Design Decisions & Rationale**
- 9. Implementation Details**
- 10. Screenshots / Results**
- 11. Testing Approach**
- 12. Challenges Faced**
- 13. Learnings & Key Takeaways**
- 14. Future Enhancements**
- 15. References**

2. Introduction

Learning is a lifelong process that extends beyond classrooms and formal education. It includes academic and professional development, reading, acquiring new skills, exploring interests, and pursuing personal goals. However, managing these different aspects of learning can become difficult when tasks, goals, reading lists, and areas of interest are scattered across different tools and platforms.

This project presents a **Personal Learning Organization System**, a Python-based application designed to bring these activities together in one place. The system allows users to manage tasks, track performance, organize their reading, create a personal curriculum, and monitor their progress.

The application is organised as a main menu that routes the user to four independent feature modules:

- **To-Do List (tasks.py)**: record tasks with a subject and deadline, mark them complete, delete them and view completion progress.
- **Subject Performance (subject.py)**: record exam marks per subject and view the marks together with the average.
- **Book Tracker (book.py)**: maintain a reading list with a status for each book, search by genre for recommendations and see reading statistics.
- **Personal Curriculum (curriculum.py)**: plan learning topics, break each into learning objectives, tick them off and track overall progress.

The project aims to provide a simple and structured way for individuals to organize their learning journey and maintain a clear overview of their goals and progress. It demonstrates the practical application of fundamental Python concepts, including functions, conditional statements, loops, collections, input handling, and modular program design.

The project is written in core Python 3 using only built-in data types (lists, dictionaries), functions, loops and modules. It deliberately avoids external libraries so that the focus stays on program design, modular structure and correct handling of user input.

Objectives

- Build a complete, working multi-module application in Python.
- Apply modular programming: one file per feature, one entry point.
- Practise CRUD operations (create, read, update, delete) on in-memory data structures.
- Provide meaningful progress and statistics so that the tool supports real study habits.

3. Problem Statement

Students often manage their academic life through scattered tools. Deadlines live in one app, marks on paper, reading lists in a notes file, and learning goals nowhere at all. This fragmentation causes missed deadlines, no visible view of performance trends, forgotten books and study goals that are never broken into achievable steps.

The problem to be solved: design and implement a single, simple, easy-to-use program that lets a student (1) organise tasks and deadlines, (2) track exam performance by subject, (3) manage a reading list, and (4) plan and monitor personal learning goals, with clear progress feedback in each area.

Scope

- **In scope:** console-based interaction, four feature modules, in-memory storage for the duration of a session, input validation for menu indices and empty data.
- **Out of scope (current version):** persistent storage, graphical interface, multiple users, authentication, reminders or notifications.

4. Functional Requirements

The functional requirements describe what the system must do. They are grouped by module and identified with FR numbers for traceability to the test cases in Section 11.

ID	Module	Requirement
FR-01	Main Menu	The system shall display a main menu with options for To-Do List, Subject Performance, Book Tracker, Personal Curriculum and Exit.
FR-02	Main Menu	The system shall return to the main menu after the user exits any module, and shall reject invalid menu choices with a message.
FR-03	To-Do List	The user shall be able to add a task with a name, subject and deadline; new tasks are marked pending.
FR-04	To-Do List	The user shall be able to view all tasks with number, name, subject, deadline and Completed/Pending status.
FR-05	To-Do List	The user shall be able to mark a selected task as complete.
FR-06	To-Do List	The user shall be able to delete a selected task.
FR-07	To-Do List	The system shall display total tasks, completed tasks and completion percentage.
FR-08	Subject Performance	The user shall be able to add a subject and delete an existing subject.
FR-09	Subject Performance	The user shall be able to view the list of subjects.
FR-10	Subject Performance	The user shall be able to enter an exam mark for an existing subject; multiple results per subject are allowed.
FR-11	Subject Performance	The system shall display, for each subject, all recorded marks and their average.
FR-12	Book Tracker	The user shall be able to add a book with title, author and genre; the default status is 'Want to Read'.
FR-13	Book Tracker	The user shall be able to view all books with their details and current status.
FR-14	Book Tracker	The user shall be able to change a book's status to Want to Read, Reading or Completed.
FR-15	Book Tracker	The system shall list books matching a genre entered by the user (case-insensitive).
FR-16	Book Tracker	The system shall display reading statistics: total, completed, currently reading and want to read.
FR-17	Book Tracker	The user shall be able to delete a selected book.
FR-18	Personal Curriculum	The user shall be able to add a learning topic with its field and the reason for studying it.
FR-19	Personal Curriculum	The user shall be able to add multiple learning objectives to a selected topic.
FR-20	Personal Curriculum	The user shall be able to view every topic with its objectives shown as Completed or Pending.
FR-21	Personal Curriculum	The user shall be able to mark an objective of a chosen topic as complete.

ID	Module	Requirement
FR-22	Personal Curriculum	The system shall display total objectives, completed objectives and overall progress percentage.
FR-23	Personal Curriculum	The user shall be able to delete a topic together with all of its objectives.
FR-24	All Modules	When a list is empty, the system shall show an informative message instead of failing.

5. Non-functional Requirements

Non-functional requirements describe the qualities the system should have rather than specific features.

ID	Category	Requirement
NFR-01	Usability	Menus shall be numbered, short and consistent across modules so that a first-time user can operate the program without instructions.
NFR-02	Maintainability	Each feature shall live in its own module with a single public function, so it can be modified or tested independently.
NFR-03	Readability	Code shall use descriptive variable names, consistent indentation and a uniform structure for every menu option.
NFR-04	Performance	All operations shall respond instantly for the expected data size (tens to hundreds of records per module).
NFR-05	Portability	The program shall run on any operating system with Python 3 installed and shall need no third-party packages.
NFR-06	Reliability	Invalid list numbers, unknown subject names and empty lists shall be handled with messages and not cause a crash.
NFR-07	Simplicity	The design shall rely only on core Python constructs (lists, dictionaries, loops, functions, modules).
NFR-08	Extensibility	A new module can be added by writing one new file and adding one menu entry in the main file.
NFR-09	Data lifetime (limitation)	Data is kept in memory for the current run only. Persistent storage is listed as a future enhancement.

6. System Architecture

The system follows a simple layered, modular architecture. The user interacts through the console. The main file (`menu.py`) acts as the presentation and routing layer. It imports one function from each feature module. Each feature module contains both its menu loop and its business logic, and owns a single module-level data structure that acts as its in-memory data store.

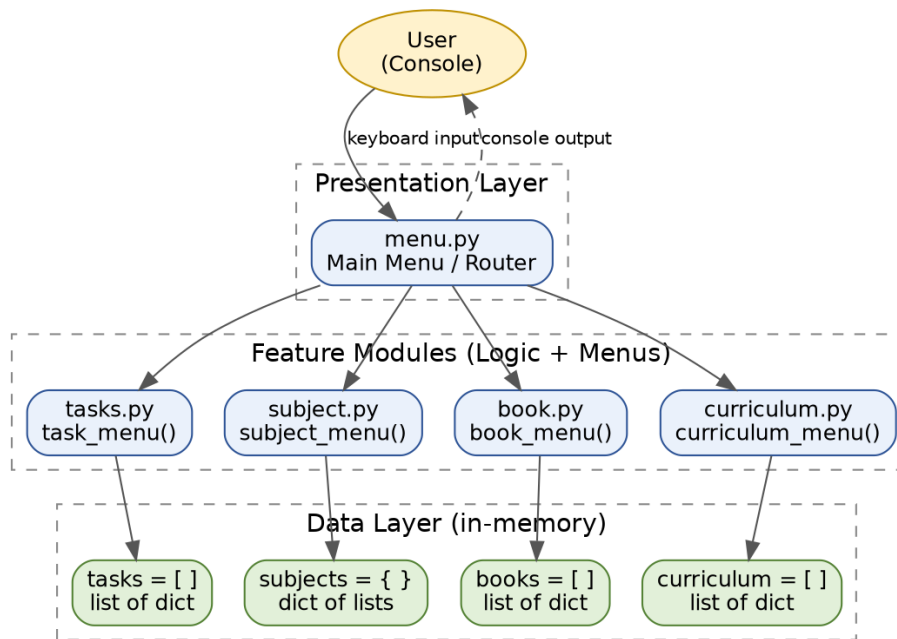


Figure 6.1: Layered architecture of the Student Productivity Manager

Layers and responsibilities

- **User / Console:** input through `input()`, output through `print()`.
- **Presentation layer (`menu.py`):** shows the main menu, reads the choice and calls the matching module function; ends the program on Exit.
- **Feature modules:** `tasks.py`, `subject.py`, `book.py` and `curriculum.py` each run their own menu loop, validate inputs and perform add / view / update / delete / statistics operations.
- **Data layer:** a list or dictionary per module (tasks, subjects, books, curriculum), created once when the module is first imported and shared for the whole session.

Technology stack

Component	Technology
Language	Python 3
Interface	Command-line (console) using <code>input()</code> and <code>print()</code>
Data storage	In-memory lists and dictionaries
Code organisation	Python modules (one .py file per feature)
Version control	Git and GitHub

Project file structure

```
Scholar-Forge/  
|-- menu.py           # entry point:  
main menu and routing  
|-- tasks.py         # To-Do List  
module  
|-- subject.py       # Subject  
Performance module  
|-- book.py          # Book Tracker  
module  
|-- curriculum.py    # Personal  
Curriculum module  
|-- README.md  
`-- report/          # this project  
report
```

7. Design Diagrams

7.1 Use Case Diagram

The only actor is the Student, who can use all four modules. Each module groups its use cases. Every module offers create, view and delete operations, plus module-specific actions such as marking completion, entering marks, changing reading status or viewing statistics.

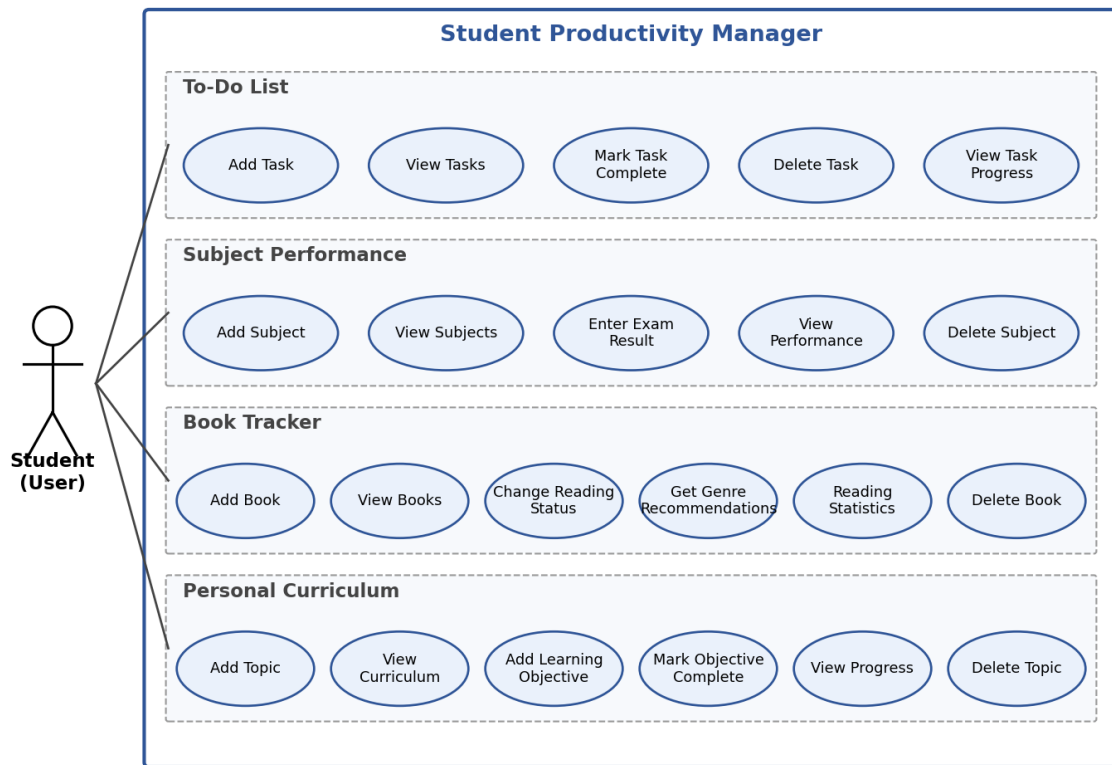


Figure 7.1: Use case diagram

7.2 Workflow Diagram

The workflow shows how control moves through the program. The user chooses a module from the main menu, then stays in that module's loop performing operations until choosing Exit, which returns to the main menu. Choosing Exit at the main menu ends the program. In every module, the input is validated first, and empty lists or invalid numbers result in a message instead of an operation.

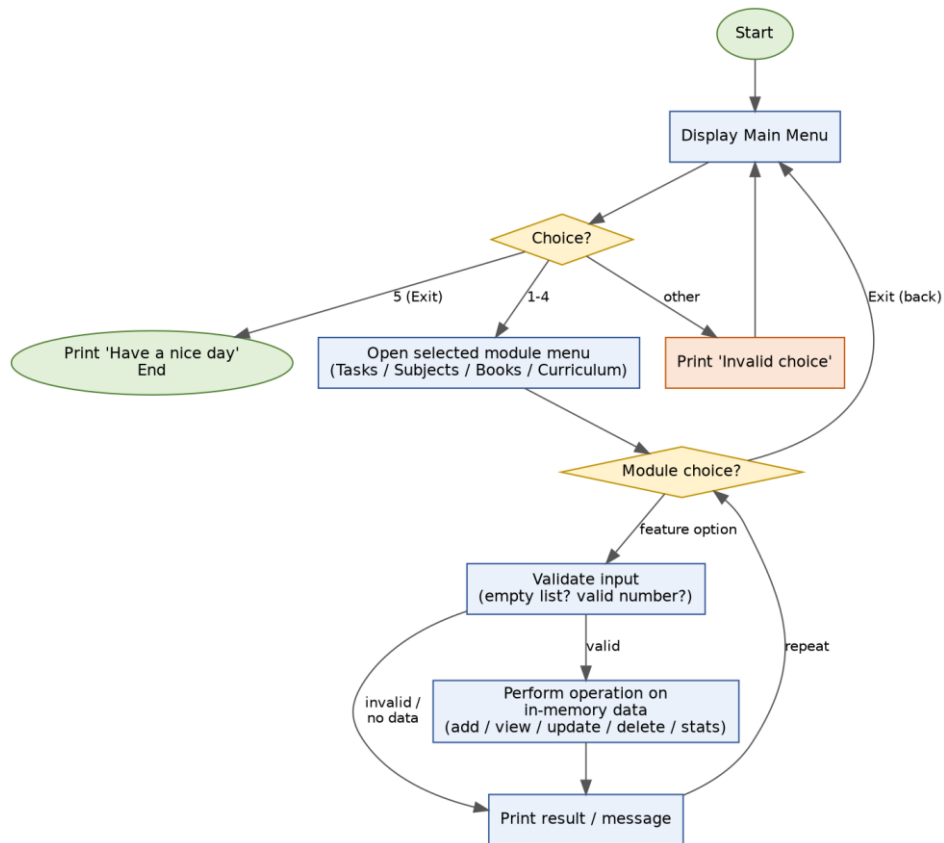


Figure 7.2: Workflow diagram

7.3 Sequence Diagram

The sequence diagram shows one representative scenario: the user opens the To-Do List, adds a task, marks a task complete and exits. The other three modules follow the same interaction pattern with their own data lists.

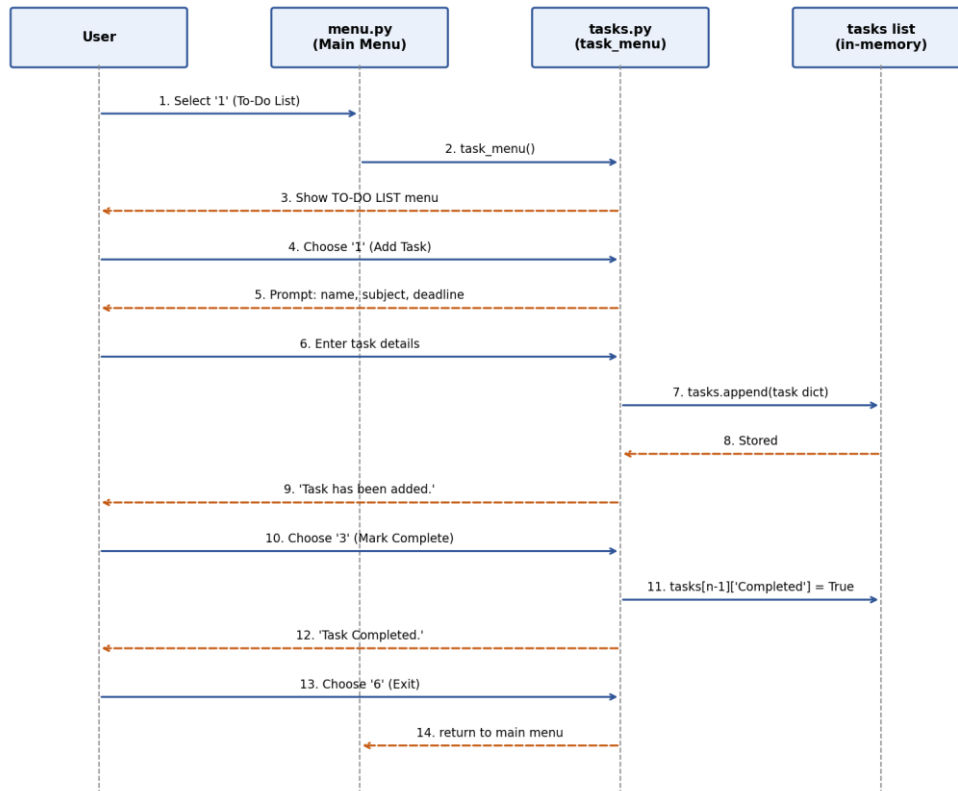


Figure 7.3: Sequence diagram for adding and completing a task

7.4 Class / Component Diagram

The program is procedural and uses modules and dictionaries, not classes. The diagram is therefore a component diagram. Each box is a module with its data store and public function. The green boxes show the structure of the dictionary records held in each list, and the diamonds show composition (a module owns its records; a Topic owns its Objectives).

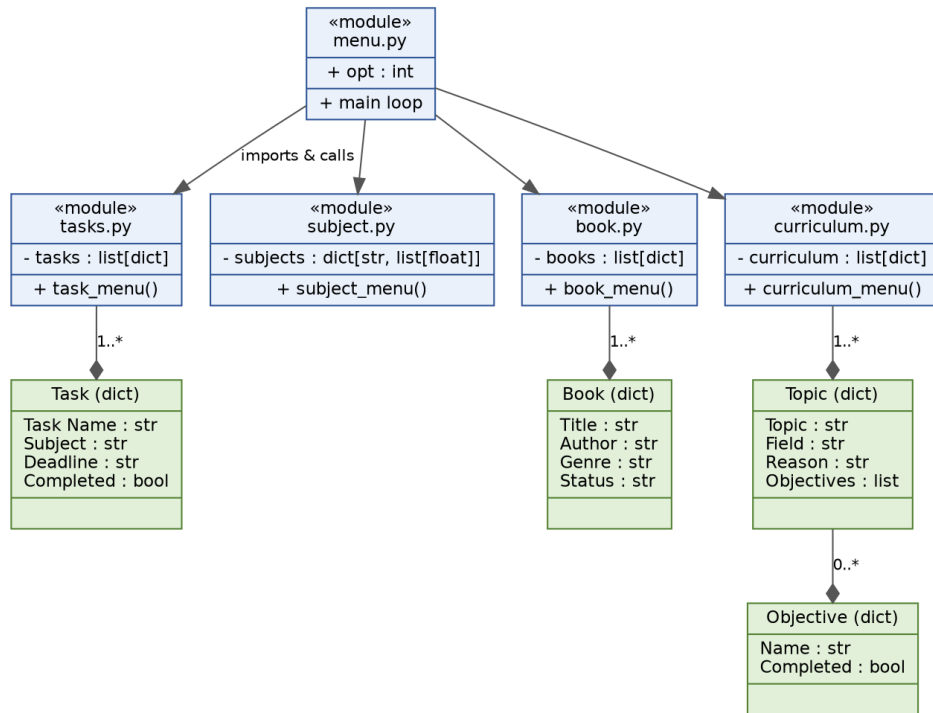


Figure 7.4: Component diagram with record structures

7.5 ER Diagram

The application does not use a database or files; all data lives in memory. The diagram below is therefore a logical data model that shows how the records relate, and it would map directly to tables if storage were added later. The only true parent-child relationship is Topic to Objective (one topic has many objectives, stored as a nested list). A Task stores its subject as free text, so its link to Subject is informal.

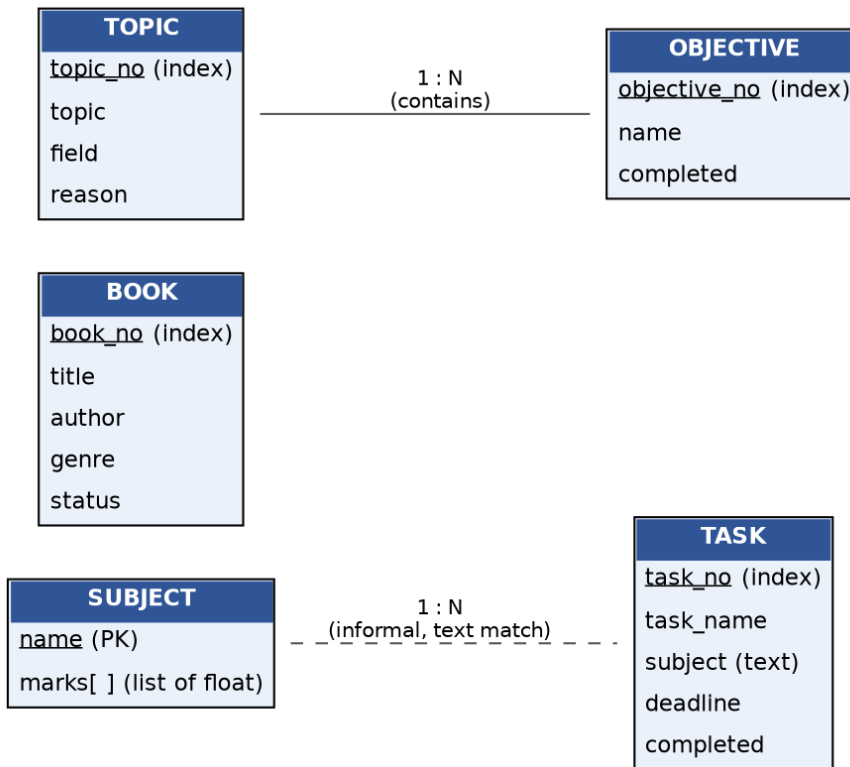


Figure 7.5: Logical ER diagram

8. Design Decisions & Rationale

Decision	What was chosen	Rationale
Program structure	Separate module per feature with one entry point (menu.py)	Keeps files short, lets each feature be developed and tested independently and mirrors how larger applications are organised.
Data storage	Module-level lists and dictionaries	The simplest structure that supports all required operations. No file handling was needed for the core learning goals.
Record format	Dictionaries with descriptive keys (e.g. "Task Name", "Status")	Readable and self-documenting compared with tuples or parallel lists, and easy to extend with new fields.
Subjects as a dictionary	<code>subjects = {name: [marks]}</code>	Subject names are unique keys and give direct lookup by name; a list of marks per subject makes averages trivial.
Curriculum structure	Topic dictionary containing a list of objective dictionaries	Reflects the real hierarchy (topic contains objectives) and allows progress to be computed across all topics.
Reading status	Three fixed text states, default 'Want to Read'	Mirrors the natural life-cycle of a book and makes the statistics simple to compute.
Navigation	Numbered menus inside while True loops with break on Exit	Familiar pattern for console programs; the loop keeps the user inside a module until they choose to leave.
Selecting items	List items with numbers starting at 1; convert to index with number - 1	Users see natural 1-based numbers while Python lists stay 0-based.
Error handling	Check for empty lists and validate number ranges before use	Avoids IndexError and gives the user a clear message instead of a crash.
Progress calculation	$\text{Percentage} = \text{completed} / \text{total} * 100$, with 0 when total is 0	Gives immediate motivational feedback and prevents division by zero.

9. Implementation Details

Every module follows the same pattern: a module-level data structure, one public function containing a while True menu loop, and an if / elif chain that maps each menu number to an operation. This consistency makes the code easy to read and to extend.

9.1 Main file: menu.py

Imports task_menu, subject_menu, book_menu and curriculum_menu, then loops over the main menu. Choices 1 to 4 call the corresponding function; choice 5 prints a goodbye message and breaks out of the loop. Any other number prints 'Invalid choice.'

9.2 To-Do List: tasks.py

Tasks are stored in the list tasks. Each task is a dictionary with the keys Task Name, Subject, Deadline and Completed (a boolean). Marking a task complete sets Completed to True; deleting uses list.pop(). The progress option counts completed tasks and derives the percentage.

```
task = {
    "Task Name": name, "Subject": subject,
    "Deadline": deadline, "Completed": False
}
tasks.append(task)

# progress
comp = sum(1 for t in tasks if t["Completed"])
progress = comp / len(tasks) * 100 if tasks else 0
```

9.3 Subject Performance: subject.py

Subjects are stored in the dictionary subjects, where the key is the subject name and the value is a list of marks (floats). Entering a result appends to that list after checking that the subject exists. The performance view loops through all subjects and prints the marks and their average, or a message if no results exist yet.

```
subjects[name] = [] # add subject
subjects[name].append(marks) # add exam result
avg = total / len(subjects[subject]) # average marks
```

9.4 Book Tracker: book.py

Books are stored in the list books as dictionaries with Title, Author, Genre and Status. New books start as 'Want to Read'. Changing status shows a sub-menu (1 Want to Read, 2 Reading, 3 Completed). Recommendations compare the entered genre with each book's genre in lower case and print matches with their author and status. Statistics count books by status.

```
if book["Genre"].lower() == genre: # case-insensitive match
    print(book["Title"])
    print("Author: ", book["Author"])
    found = True
```

9.5 Personal Curriculum: curriculum.py

Topics are stored in the list curriculum. Each topic dictionary has Topic, Field, Reason and an Objectives list of dictionaries with Name and Completed. Marking an objective complete is a two-step selection: first the topic, then the objective inside it. Progress is calculated across all objectives of all topics using nested loops.

```
objective = {"Name": obj_name, "Completed": False}
curriculum[num - 1]["Objectives"].append(objective)

for topic in curriculum:
    for objective in topic["Objectives"]:
        total += 1
        if objective["Completed"]:
            comp += 1
```

9.6 Reusable patterns used across modules

- **Empty check:** if len(items) == 0: print("No ... available.") before any listing or selection.
- **Numbered selection:** enumerate(items, start=1) to display, then range check 1 <= num <= len(items) before using num - 1.
- **Menu loop:** while True with break on the Exit option and a final else branch for invalid choices.

10. Screenshots / Results

The boxes below show the expected output for the key features.

10.1 Main menu

```
1. To-Do List
2. Subject Performance
3. Book Tracker
4. Personal Curriculum
5. Exit

Enter your choice:
```

Figure 10.1: Main menu

10.2 To-Do List: adding and viewing tasks

```
TO-DO LIST

1. Add Tasks
2. View Tasks
3. Mark Task Complete
4. Delete Task
5. View Task Completion Progress
6. Exit
Enter your choice: 1
Task name: Make Python Project File
Subject: CSE
Deadline: 30/09/2026
Task has been added.
TO-DO LIST

1. Add Tasks
2. View Tasks
3. Mark Task Complete
4. Delete Task
5. View Task Completion Progress
6. Exit
Enter your choice: 2
Number of Task: 1
Name: Make Python Project File
Subject: CSE
Deadline: 30/09/2026
Status: Pending
```

Figure 10.2: Adding and viewing a task

```

TO-DO LIST

1. Add Tasks
2. View Tasks
3. Mark Task Complete
4. Delete Task
5. View Task Completion Progress
6. Exit

Enter your choice: 3
1 Make Python Project File
Enter task number: 1
Task Completed.
TO-DO LIST

1. Add Tasks
2. View Tasks
3. Mark Task Complete
4. Delete Task
5. View Task Completion Progress
6. Exit

Enter your choice: 5
Total tasks: 1
Completed: 1
Progress: 100.0 %

```

Figure 10.3: Marking a task complete and viewing progress

10.3 Subject Performance

```

SUBJECT PERFORMANCE

1. Add Subject
2. View Subjects
3. Enter Exam Result
4. View Performance
5. Delete Subject
6. Exit

Enter your choice: 1
Subject name: CSE
Subject has been added.
SUBJECT PERFORMANCE

1. Add Subject
2. View Subjects
3. Enter Exam Result
4. View Performance
5. Delete Subject
6. Exit

Enter your choice: 3
CSE
Enter subject name: CSE
Enter marks: 89
Result has been added.
SUBJECT PERFORMANCE

1. Add Subject
2. View Subjects
3. Enter Exam Result
4. View Performance
5. Delete Subject
6. Exit

Enter your choice: 4
Subject: CSE
Exam Results: [89.0]
Average Marks: 89.0

```

Figure 10.4: Exam results and average marks

10.4 Book Tracker

```
BOOK TRACKER

1. Add Book
2. View Books
3. Change Reading Status
4. Book Recommendations
5. Reading Statistics
6. Delete Book
7. Exit

Enter your choice: 1
Book title: The Secret History
Author: Donna Tartt
Genre: Psychological Thriller
Book has been added.
BOOK TRACKER

1. Add Book
2. View Books
3. Change Reading Status
4. Book Recommendations
5. Reading Statistics
6. Delete Book
7. Exit

Enter your choice: 3
1 The Secret History
Enter book number: 1

1. Want to Read
2. Reading
3. Completed

Enter status: 2
Reading status updated.
```

Figure 10.5: Changing reading status and viewing statistics

```
BOOK TRACKER

1. Add Book
2. View Books
3. Change Reading Status
4. Book Recommendations
5. Reading Statistics
6. Delete Book
7. Exit
Enter your choice: 4
Enter genre: Psychological Thriller
The Secret History
Author: Donna Tartt
Status: Reading
BOOK TRACKER

1. Add Book
2. View Books
3. Change Reading Status
4. Book Recommendations
5. Reading Statistics
6. Delete Book
7. Exit
Enter your choice: 5
Total Books: 1
Completed: 0
Currently Reading: 1
Want to Read: 0
BOOK TRACKER
```

Figure 10.6: Recommendations by genre (case-insensitive)

10.5 Personal Curriculum

```

PERSONAL CURRICULUM

1. Add Learning Topic
2. View Curriculum
3. Add Learning Objective
4. Mark Objective Complete
5. View Progress
6. Delete Learning Topic
7. Exit
Enter your choice: 2
Topic Number: 1
Topic: The Philosophy of Art
Field: Art
Reason: To see why art exists, how emotions, imagination, express itself in the form of art. What does it hide?
Objectives:
Completed: Art and truth
Pending: Art and morality
Pending: Art and emotion
PERSONAL CURRICULUM

1. Add Learning Topic
2. View Curriculum
3. Add Learning Objective
4. Mark Objective Complete
5. View Progress
6. Delete Learning Topic
7. Exit
Enter your choice: 5
Total Objectives: 3
Completed Objectives: 1
Progress: 33.33333333333333 %

```

Figure 10.7: Curriculum view and overall progress

10.6 Handling of empty data and invalid input

```

1. To-Do List
2. Subject Performance
3. Book Tracker
4. Personal Curriculum
5. Exit

Enter your choice: 1
TO-DO LIST

1. Add Tasks
2. View Tasks
3. Mark Task Complete
4. Delete Task
5. View Task Completion Progress
6. Exit
Enter your choice: 2
No tasks available.
TO-DO LIST

1. Add Tasks
2. View Tasks
3. Mark Task Complete
4. Delete Task
5. View Task Completion Progress
6. Exit
Enter your choice: 7
Invalid choice.

```

Figure 10.8: Friendly messages for empty lists and invalid numbers

11. Testing Approach

The application was tested manually using black-box functional testing: each menu option was run with valid input, boundary values and invalid input, and the console output was compared with the expected result. Testing was repeated after every change to a module to make sure earlier features still worked (regression testing). Modules were first tested individually and then together through the main menu to check navigation.

- **Positive tests:** valid data confirms each feature does what the requirement states.
- **Negative tests:** empty lists, out-of-range numbers and unknown names confirm the program responds with a message.
- **Boundary tests:** first and last item of a list, zero objectives, zero tasks (division by zero in progress).
- **Integration tests:** moving between the main menu and each module and back repeatedly.

ID	Req.	Test scenario / input	Expected result	Status
T-01	FR-01	Choose 1 to 4 at main menu	Matching module menu is shown	Pass
T-02	FR-02	Choose 9 at main menu	'Invalid choice.' and menu re-displays	Pass
T-03	FR-03	Add task with name, subject, deadline	'Task has been added.'; status Pending	Pass
T-04	FR-24	View tasks when list is empty	'No tasks available.'	Pass
T-05	FR-05	Mark task 1 complete	Status shows Completed	Pass
T-06	FR-06	Delete task number 99	'Invalid Task Number.'; nothing removed	Pass
T-07	FR-07	Progress with 0 tasks	0 total, 0 completed, 0 %; no crash	Pass
T-08	FR-10	Enter marks for unknown subject	'Subject not found.'	Pass
T-09	FR-11	Marks 78 and 86 for Math	Results [78.0, 86.0]; average 82.0	Pass
T-10	FR-14	Change book status to 3	Status becomes Completed	Pass
T-11	FR-14	Change book status to 7	'Invalid status.'; status unchanged	Pass
T-12	FR-15	Genre entered as 'FICTION' for genre 'Fiction'	Book is listed (case-insensitive)	Pass
T-13	FR-15	Genre with no matching book	'No books found for this genre.'	Pass
T-14	FR-21	Mark objective in a topic with no objectives	'No objectives available.'	Pass

ID	Req.	Test scenario / input	Expected result	Status
T-15	FR-22	2 objectives, 1 completed	Progress 50.0 %	Pass
T-16	FR-23	Delete a topic that has objectives	Topic and its objectives removed	Pass
T-17	NFR-06	Type text (e.g. abc) where a number is expected	Not handled: program stops with ValueError	Known limitation
T-18	FR-08	Add a subject name that already exists	Not handled: existing marks are overwritten	Known limitation

Tests T-17 and T-18 document known limitations that are listed under Future Enhancements.

12. Challenges Faced

- **1-based display versus 0-based lists.** Users should see item numbers starting at 1, but Python indexes start at 0. Using `enumerate(..., start=1)` for display and `num - 1` for access, together with a range check, solved this and prevented off-by-one errors and `IndexError` crashes.
- **Nested data in the curriculum module.** A topic contains objectives, so marking an objective complete needs a two-level selection. Storing objectives as a list inside each topic dictionary and selecting the topic first made the logic manageable.
- **Division by zero in progress calculations.** With no tasks or objectives the percentage formula would fail. An explicit check for zero total before dividing fixed this.
- **Organising code across modules.** Splitting the program into separate files required each module to keep its own data and expose one function, and the main file to import them correctly, so that data persists across repeated visits to a module within one run.
- **Consistent menu behaviour.** Keeping every module's menu, messages and exit behaviour uniform needed careful, repeated patterns and distinct variable names for each menu choice (`opt`, `opt_1`, `opt_2`, `opt_3`, `opt_4`).
- **Handling empty and invalid states.** Every operation needed a different message for 'no data', 'invalid number' and 'not found', which made the branching deep and highlighted the value of helper functions.
- **Data lost on exit.** Because storage is in memory, all records vanish when the program closes. This was accepted as a scope limit for this version and became the top future enhancement.

13. Learnings & Key Takeaways

- Modular programming makes a larger program easy to organise: one file per feature, imported and called from a single entry point.
- Lists and dictionaries are powerful enough to model real data, including nested structures such as topics with objectives.
- CRUD operations (create, read, update, delete) are the backbone of most applications, and the same pattern repeats in every module.
- Defensive programming matters: checking for empty data and invalid indices before using them removes most runtime errors.
- Repeated code is a signal for refactoring; helper functions for selection and validation would reduce duplication considerably.
- Planning with diagrams (use case, workflow, sequence, component) before and during coding clarifies the design and makes documentation easier.
- Testing edge cases (empty lists, zero totals, invalid choices) finds problems that ordinary use never shows.
- Version control with GitHub supports incremental development and safe experimentation.

14. Future Enhancements

Enhancement	Description
Persistent storage	Save and load data using JSON files (or SQLite) so records survive between sessions. This would also make the ER diagram a real database schema.
Robust input validation	Wrap number input in try / except ValueError so that non-numeric input asks again instead of crashing; validate marks range (for example 0 to 100) and date format for deadlines.
Duplicate prevention	Reject subject names that already exist and normalise case and whitespace ('Math' and 'math' treated as the same).
Link tasks to subjects	Let users pick a task's subject from the existing subjects list and view task progress per subject.
Deadline intelligence	Use the datetime module to sort tasks by deadline, highlight overdue tasks and show what is due today or this week.
Refactoring	Extract repeated list, select and validate code into shared helper functions; consider classes for Task, Book, Topic and Subject.
Delete confirmation and edit	Ask for confirmation before deleting and allow editing existing entries rather than only delete and re-add.
Richer analytics	Grade trends across exams, highest and lowest subjects, reading goals per month, and charts using a library such as matplotlib.
Graphical / web interface	Move from console to a Tkinter desktop app or a web app (for example Flask) for a friendlier experience.
Multi-user and reminders	User accounts and notifications or email reminders for approaching deadlines.

15. References

- [1] Python Software Foundation. The Python Tutorial: Data Structures (lists and dictionaries). <https://docs.python.org/3/tutorial/datastructures.html>
- [2] Python Software Foundation. The Python Tutorial: Modules. <https://docs.python.org/3/tutorial/modules.html>
- [3] Python Software Foundation. The Python Tutorial: Input and Output. <https://docs.python.org/3/tutorial/inputoutput.html>
- [4] Python Software Foundation. The Python Tutorial: Errors and Exceptions. <https://docs.python.org/3/tutorial/errors.html>
- [5] GitHub Docs. Getting started with Git and GitHub. <https://docs.github.com>
- [6] Graphviz documentation (used to draw the architecture, workflow, component and ER diagrams). <https://graphviz.org/documentation/>
- [7] Class notes and Modules.
- [8] GeeksforGeeks, Python Programming Tutorials
- [9] VITyarthi Python essential course